

Scikit learn

Classify flowers on Colab

Machine Learning Workflow

What are the steps of a machine learning project?

1. load data (always use pandas and csv)
2. explore data:
 - a. min, avg, percentiles, missing, categories, correlation, ...
3. transform data:
 - a. clean data, encode data, engineer features
4. choose sklearn model
 - a. start simple with logistic / linear regression to get a benchmark)
5. build X and y : design matrix and target variable
6. split the data into a train and a test subset
7. train the model with the fit() method on the train subset
8. evaluate the score on the test subset

Google Colab

Working on Google Colab

- Zero Setup: no local install
- Collaboration: shareable, promotes reproducibility
- Provides free access to GPUs and TPUs
- Pre-Installed Libraries: sklearn, keras, etc
- Integration with Google Drive
- Free

<https://colab.research.google.com/>

Open a google colab notebook

- connect to a runtime
- create a python cell, execute it
- create a markdown cell
 - #, ##, ###, word, **bold**, underline
- add a file to the notebook
- add an API key
- install a package with `!pip install package_name`
- what else ?

SQRT(2)

The SQRT(2) algorithm

Invented by [Heron](#) a greek mathematician in the 1st century

You want to calculate the square root of 2. (works with any number)

1. start with an initial value $x = 1 < 2$
2. update x : $x = x + x/2$
3. until x is near enough the expected result i.e. x^2 close to 2

In short, with precision a small value ($p \sim 0.01$)

```
While  $|x^2 - 2| > \text{precision}$ 
```

```
 $x = (x + 2/x)/2$ 
```

This convergences super fast.

The number of correct digits of x roughly doubles with each iteration.

Heron's method in python

Open a colab notebook

- code Heron's method
- set the error as a separate variable
- change the error from $\text{abs}(x^2 - 2)$ to $\text{sqrt}((x^2 - 2)^2)$
- in both cases plot the error, the log of the error vs the number of iterations

ML 2000 years ago

In Heron's method we have

- the prediction error
- the ground truth
- convergence
- a loss function
- iterations

Pandas

Pandas

The best thing since e-bikes, smart phones and wifi
you ~~can~~ should do all your data manipulation, transformation in pandas

- load and export csv files
 - explore data
 - find anomalies, clean up data
 - create new columns, encode
-
- no loop needed
 - some quirks but you'll get used to them
 - works well with numpy and sklearn

Pandas tutorial

A 10 mn tutorial by
Wes McKinney
creator of pandas

```
2000-01-08 04:00:00      NaN      NaN
2000-01-08 08:00:00      NaN      NaN
2000-01-08 12:00:00      NaN      NaN
2000-01-08 16:00:00      NaN      NaN
2000-01-08 20:00:00      NaN      NaN
2000-01-08 00:00:00  8.621586  0.621586
2000-01-09 04:00:00      NaN      NaN
2000-01-09 08:00:00      NaN      NaN
2000-01-09 12:00:00      NaN      NaN
2000-01-09 16:00:00      NaN      NaN
2000-01-09 20:00:00      NaN      NaN
2000-01-10 00:00:00  1.862927      NaN

In [274]: df.resample('4h').first(limit=3)

Out[274]:
```

	A	B
2000-01-01 00:00:00	-0.030009	-0.030009
2000-01-01 04:00:00	-0.030009	-0.030009
2000-01-01 08:00:00	-0.030009	-0.030009
2000-01-01 12:00:00	-0.030009	-0.030009
2000-01-01 16:00:00	NaN	NaN
2000-01-01 20:00:00	NaN	NaN
2000-01-02 00:00:00	1.297861	NaN
2000-01-02 04:00:00	1.297861	NaN
2000-01-02 08:00:00	1.297861	NaN
2000-01-02 12:00:00	1.297861	NaN
2000-01-02 16:00:00	NaN	NaN
2000-01-02 20:00:00	NaN	NaN
2000-01-03 00:00:00	-0.005490	-0.005490

Practice pandas

notebooks with exercises in pandas

- go on Colab
- Do the notebooks
 - `pandas_exercises_gemini_01.ipynb`
 - `pandas_exercises_gemini_02.ipynb`
- and the tutorial notebook from google colab
 - [Pandas DataFrame UltraQuick Tutorial.ipynb - Colab](#)

What are the steps of a machine learning project?

- ~~1. load data (always use pandas and csv)~~
- ~~2. explore data~~
- ~~3. transform data~~
4. choose sklearn model
 - a. start simple with logistic / linear regression to get a benchmark)
5. build X and y : design matrix and target variable
6. split the data into a train and a test subset
7. train the model with the fit() method on the train subset
8. evaluate the score on the test subset

Scikit-learn

Scikit-learn

History

- Production-ready machine learning for non-deep learning tasks
- Developed in 2007 as a Google Summer of Code project
- Built on NumPy, SciPy, and Matplotlib
- Version 1.0 released in 2021, marking API stability
- Open source: Maintained by a global community of volunteers

Used for

- Supervised Learning: Classification & Regression
- Unsupervised Learning: Clustering & Dimensionality Reduction
- Model Selection: Cross-validation & Hyperparameter Tuning
- Feature Engineering: Preprocessing & Feature Selection

<https://scikit-learn.org/stable/>

Linear Models

https://scikit-learn.org/stable/modules/linear_model.html

- import model
- instantiate model
 - a model corresponds to one or several loss functions
- model parameters
 - models specific params (learning rate, batch size, ...)
 - sklearn params (njobs, ...)
- model attributes
- methods
 - fit() : train the model
 - predict() : inference on unseen data
 - score() : performance evaluation



- Open source, 2010
- Basé sur numpy, scipy, ...
- Inria, telecom parisTech, NYU,
- Olivier Grisel, Andreas Muller, Gael Varoquaux, Jake Vanderplas
- Autres librairies sur le meme concept: astropy, ...



Scikit-learn

Algorithmes et modeles:

- Supervision, classification et regression
- Clustering
- Reduction de dimensionnalite

Mais aussi:

- Selection de modele
- Preprocessing
- Feature selection

Input:

- Images, text, numerique, ...

Scikit-learn

- En input
 - X: matrice de predicteurs
 - y: vecteur cible
 - Methode
 - clf =
 - clf.fit
 - clf.predict
 - clf.score
- [Modeles lineaires](#)
 - [SGD](#)
 - [Random Forest](#)
 - [Comparaison de classifieurs](#)

Linear Regression

[LinearRegression — scikit-learn 1.5.2 documentation](#)

LinearRegression

```
class sklearn.linear_model.LinearRegression(*, fit_intercept=True,  
copy_X=True, n_jobs=None, positive=False)
```

[\[source\]](#)

Ordinary least squares Linear Regression.

LinearRegression fits a linear model with coefficients $w = (w_1, \dots, w_p)$ to minimize the residual sum of squares between the observed targets in the dataset, and the targets predicted by the linear approximation.

Titanic

Scikit-learn on titanic

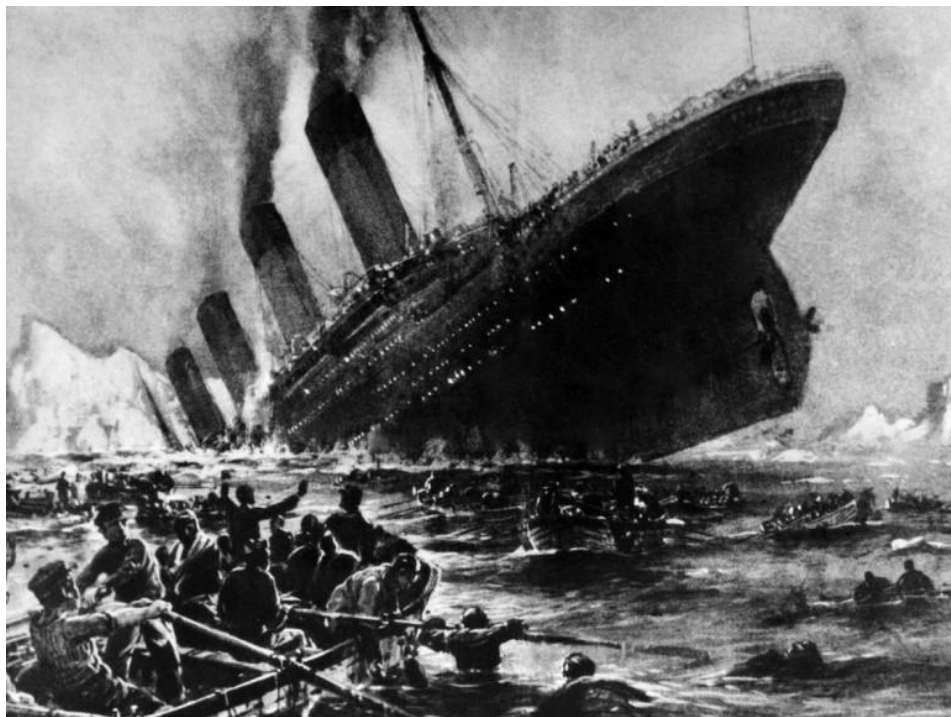
Let's train a simple classifier on the titanic.

The *getting started* competition on [Kaggle](#)

We want to predict whether a passenger will survive or not.

The dataset has 1300 rows of passengers with

- name and title
- pclass: 1st to 3rd class
- age,
- sex: male or female
- fare: price paid for the ticket
- sibsp: # of siblings / spouses aboard
- parch: # of parents / children aboard
- embarked: Port of Embarkation
- home.dest



How to Get Started with Kaggle's Titanic Machine Learning Competition

kaggle

titanic with simple LogisticRegression

[LogisticRegression — scikit-learn 1.5.2 documentation](#)

Load the notebook `titanic_gemini.ipynb` and follow along

Iris with simple LogisticRegression

[LogisticRegression — scikit-learn 1.5.2 documentation](#)

Now create a new notebook and train a model on the iris dataset

- load the iris dataset into a pandas dataframe
- encode the target variable ($f = 1, m = 0$)
- import a LogisticRegression classifier
- train the model with `fit()`
- evaluate the performance with accuracy and confusion matrix
- plot the histogram of predicted probabilities